生成 AI に伝わりやすい Markdown の書き方

意図が伝わる Markdown 指示の作り方

(株) SRA

2026-11-13

生成 AI に伝わりやすい Markdown の書き方

Markdown は、生成 AI への指示を構造化して誤解を減らすために使えます。特に、見出し、箇条書き、番号付き手順、コードブロックを明確に使うことが重要です。

この文書は、人が読む依頼文や指示文を Markdown で書くときのコツをまとめたものです。Markdownは意図を整理するための形式であり、指示の遵守や出力の正確さを保証するものではありません。機械可読な構造化出力が必要な場合は、JSON Schema などの structured output を使う方が適していることがあります。

1. 見出しで話題を分ける

`#` や `##` は、指示の種類を分けるために使います。

```
# 依頼内容
## 目的
## 対象ファイル
## 変更内容
## 注意事項
```

見出しがあると、生成 AI は「何をしてほしいか」「どこを触るか」「何に注意すべきか」を区別しやすくなります。

2. 箇条書きで要件を並べる

`-` は、満たすべき条件を列挙するときに使います。

```
- `docs/README.md` を編集する
- 新しいファイルは作らない
- 箇条書きは 1 行で書く
- 必要なければフルパスは書かない
```

要件を項目ごとに分けると、人もAIも条件を確認しやすくなります。項目があることと、実際に満たしたことは別に確認します。

3. 番号付きリストで順序を示す

`1.` `2.` `3.` は、手順や優先順位を指定するときに使います。

```
1. `docs` 配下の `.md` を確認する
2. `README.md` を作成する
3. 各ファイルを 1 行で説明する
4. 重複表現を削る
```

順番が重要な場合は、箇条書きより番号付きの方が明確です。

4. コードブロックで出力例や完成イメージを示す

複数行の具体例や出力例はコードブロックで囲みます。

````
```md
## ファイル一覧

- `release-plan.md`: リリース方針のメモ
- `target-audience.md`: ターゲット層の整理
```
````

コードブロックには、入力資料・実行するコマンド・出力例のどれなのかを、前後の文章や見出しで明示します。コードブロックで囲んだだけでは役割は決まりません。生成 AI にそのまま使ってほしい文面を示す場合にも有効です。

5. 命令と文脈を分ける

指示文と、参考資料や入力文は分けて書きます。

````
```md
## 指示

- 次の文章を 3 行で要約する
- 箇条書きは使わない

## 入力

"""
ここに要約対象の文章を書く
"""
```
````

見出しや区切り記号は、命令と入力資料を区別する手掛かりになります。ただし、入力資料中の指示を無視させる安全上の境界にはなりません。外部資料の扱いは、利用する環境の権限設定や確認手順と合わせて考えます。

6. バッククォートで対象を明確にする

ファイル名、コマンド、識別子はバッククォートで囲みます。

```
- `README.md`
- `docs/`
- `release-plan.md`
```

これで、説明文と対象物が混ざりにくくなります。

7. 曖昧な表現を避ける

次のような表現は曖昧です。

```
- いい感じに直してください
- 適当にまとめてください
- 必要なら変えてください
```

代わりに、条件を具体化します。

```
- 各 `.md` を 1 行で説明する
- ファイル名はバッククォートで囲う
- リンクが必要なら URL をそのまま書く
- セクションは 4 つまでにする
```

8. 禁止事項を明示する

してほしくないことは、はっきり書きます。

```
- 新規ディレクトリは作らない
- 既存の本文は書き換えない
- Markdown リンクは使わない
```

禁止条件は、要件と分けて書くと伝わりやすいです。

禁止だけでなく、代わりにどうしてほしいかも一緒に書くと、生成 AI は動きやすくなります。

```
- Markdown リンクは使わない
- リンクが必要なら URL をそのまま書く
```

9. 完成形を先に示して形式を合わせる

出力の形を先に見せると、生成 AI は合わせやすくなります。

````
```md
## ファイル一覧

- `a.md`: 説明
- `b.md`: 説明
- `c.md`: 説明
```
````

「どういう形式で出してほしいか」があると、意図がずれにくくなります。

10. 立場と確認する観点を示す

生成 AI に、どの立場で答えるかを先に示すと、語調や観点がぶれにくくなります。

```
- 編集者としてレビューする
- 初心者向け講師として説明する
- 厳しめのレビュアーとして問題点だけ挙げる
```

立場を指定しても、専門知識や調査・検証の裏付けを保証するものではありません。目的、対象、要件、禁止事項、出力形式も合わせて書くと、より正確に伝わります。

11. 発散を防ぐ要点をまとめる

生成 AI の発散を防ぎたいときは、次の点をまとめて書くと有効です。

```
- 目的を書く
- 対象を書く
- 作業範囲を書く
- 出力形式を書く
- 禁止事項を書く
- 代わりにどうしてほしいかを書く
```

「何をするか」「どこまでやるか」「どう出すか」「何をしないか」を同時に示すと、生成 AI は意図から外れにくくなります。

12. 良い指示の例

次は、対象・制約・完了時の確認を含む依頼の例です。

````
# 依頼

`docs/README.md` を作成してください。

## 目的

`docs` 配下の文書の役割を一覧できるようにする。

## 対象と要件

- 対象は `docs/` 直下の `.md`。既存の `README.md` 自身は一覧から除く
- 各ファイルを読み、役割を1項目で説明する
- ファイル名はバッククォートで囲い、日本語で書く
- ファイル名は重複させず、実際の内容に合わせてセクションを分ける
- 完了時に、確認したファイルと未確認のファイル、変更内容を報告する

## 制約

- 既存の `docs/README.md` がある場合は内容と未コミット変更を確認し、必要な部分だけ更新する
- その他のファイルは変更しない
- 読めなかった文書の役割を推測で埋めない
- コミットや外部への公開は行わない

## 出力イメージ

以下の名前と分類は形式を示すための架空例です。実際のファイルへ置き換えてください。

```md
## 企画資料

- `release-plan.md`: リリース方針をまとめた文書
- `target-audience.md`: 想定ユーザーを整理した文書
```
````

人も、報告された対象と実際のファイル、変更差分を照合します。「確認しました」という返答だけで、完了と判断しないようにします。

13. 補足: 生成 AI への指示で使いやすいその他の記号

生成 AI への指示では、次のようなその他の記号を使うと構造が伝わりやすくなります。

`:` は、項目名と内容を対応づけ、意味のまとまりを短く示しやすくします。

```
目的: 要点を 3 行でまとめる
```

`"` や `"""` は、入力文や引用範囲を囲み、どこまでが対象文かを伝えやすくします。

```
"""この文章を要約する"""
```

`<...>` は、タグ風のまとまりを作り、命令や入力の役割を分けやすくします。

```
<instructions>3 行で要約する</instructions>
```

`>` は、引用や補足を本文と分け、参照部分を目立たせやすくします。

```
> この段落は参考資料
```

YAML front matter の `---` は、文書先頭に設定情報をまとめ、本文とメタ情報を分離しやすくします。

```
---
title: サンプル文書
---
```

`[]()` は、リンクを明示し、参照先をそのまま示しやすくします。

```
[OpenAI](https://openai.com/)
```

`![]()` は、画像を埋め込み、参照画像を共有しやすくします。

```
![概要図](images/overview.png)
```

`|` は、表を作り、項目を行と列で比較しやすくします。

```
| 項目 | 内容 |
| --- | --- |
| 目的 | 要約する |
```

`*` `_` `**` は、強調を付け、注目してほしい語を目立たせやすくします。

```
**重要**
```

ただし、記号を増やしすぎると読みにくくなるため、目的に合うものだけを使うのが有効です。